

Requirement Analysis – Mastering the Software Requirements Specification (SRS)



UNIVERSITY OF
MORATUWA

ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Objectives

- To understand how to craft, validate, and track a high-quality SRS.
- We will cover the following sections:
 1. Elicitation & Documentation: Capturing Functional and Non-Functional Requirements.
 2. Validation & Justification: Ensuring Requirements are 'Right' and 'Necessary'.
 3. Traceability & Verification: Linking Requirements to Implementation.



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

What is an SRS?

- The software requirements specification document is the official statement of what is required of the system developers.
- Should include both a definition of user requirements and a specification of the system requirements.
- It is **NOT** a design document. As far as possible, it should say **WHAT** the system should do rather than **HOW** it should do it.



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Elicitation and Documentation

Before documenting, we must gather the requirements.

Elicitation Techniques

- **Interviews: One-on-one sessions with stakeholders.**
- **Workshops/JAD: Collaborative sessions to resolve conflicts.**
- **Observation: Watching users perform tasks (implicit requirements).**
- **Document Analysis: Reviewing legacy systems or regulations.**



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Defining Functional Requirements

- The 'What': Behaviors or functions of the system.
- Format: 'The system shall...'
- Examples:
 - 'The system shall allow users to reset their password via email.'
 - 'The system shall calculate sales tax based on the shipping address.'
- Key Distinction: FRs describe inputs, behavior, and outputs.



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Writing 'Good' Functional Requirements

- **Atomic:** Each statement describes a single function.
- **Unambiguous:** Only one interpretation (Avoid: 'user-friendly', 'fast').
- **Verifiable:** Can you write a test case for it?
- **Consistent:** Does not contradict other requirements.



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Structuring the Functional Requirements Section

- **Organization Strategies:**
 - **By User Class:** (e.g., Admin functions, Customer functions).
 - **By Feature:** (e.g., Checkout, Profile Management).
 - **By Stimulus:** (e.g., Reporting, Real-time monitoring).
- **Tip: Use unique identifiers (e.g., FR-001) for traceability.**



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Narrative Formats: Use Cases & User Stories

- User Stories (Agile focus):
 - 'As a [role], I want [feature], so that [benefit].'
*"As a **Marketing Manager**, I want to **export weekly sales reports to PDF**, so that I can **present clear performance metrics at the Monday board meeting**."*
- Use Cases (Formal detail):
 - Actors: Who is involved?
 - Pre-conditions: What must happen before?
 - Main Flow: Step-by-step success path.
 - Alternate Flows: Error handling or branches.



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Defining Non-Functional Requirements (NFRs)

- The 'How': Quality attributes or constraints.
- Importance: A system can work (Functional) but be unusable (Non-Functional).
- The 'URPS' in the FURPS+ model for classifying software requirements:
 - Usability
 - Reliability
 - Performance
 - Supportability



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Examples of NFRs

- **Performance:** 'The system shall load the dashboard in $< 2s$ for 95% of requests.'
- **Security:** 'All data at rest must be encrypted using AES-256.'
- **Scalability:** 'The system must support 10,000 concurrent users.'
- **Note:** NFRs must be measurable.



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Validation vs. Verification

- **Validation: 'Are we building the RIGHT product?' (Meets customer needs).**
- **Verification: 'Are we building the product RIGHT?' (Meets the spec/design).**
- **This section focuses on Validation.**
- **Validation Techniques**
 - **Requirements Reviews: Peer/stakeholder reviews to find errors early.**
 - **Prototyping: Building a UI mockup to validate user flows.**
 - **Model Validation: Checking diagrams (ERD, State Machines) for logic.**
 - **Acceptance Criteria Definition: Defining exactly what 'done' looks like.**



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Justification: Is this Requirement Necessary?

- **Avoid 'Gold Plating' (unnecessary features).**
- **Prioritization Techniques:**
 - **MoSCoW Method: Must, Should, Could, Won't have.**
- **Cost/Benefit Analysis: Does business value justify dev cost?**
- **Regulatory/Compliance: Is it legally required? (Automatic justification).**



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

What is Traceability?

- Traceability is linking requirements to other artifacts (Design, Code, Tests).
- Forward Traceability: Requirement -> Design -> Code -> Test.
- Backward Traceability: Code -> Requirement.
- Goal: Ensure 100% coverage and effective change management.

The Requirements Traceability Matrix (RTM)

Req ID	Description	Design Ref	Test Case ID	Status
FR-01	Login	DD-Sec-01	TC-001, TC-002	Pass
FR-02	Export PDF	DD-Rep-05	TC-055	Fail



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Evaluating Against Implementation

- **Verification:** Running the test cases defined in the RTM.
- **Defect Density:** High defects in a specific FR area indicate issues.
- **Gap Analysis:**
 - Identifying implemented features **NOT** in SRS (Scope Creep).
 - Identifying SRS items **MISSING** in code.



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Common Issues in SRS

- **Ambiguity:** Using vague words ('appropriate', 'user-friendly').
- **Implementation Bias:** Describing HOW code works instead of WHAT it does.
- **Untestable Requirements:** Cannot be proven true or false.
- **Lack of Traceability:** Losing track of requirements.



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Summary & Best Practices

- **Collaborate:** Elicit from multiple sources.
- **Standardize:** Use a consistent template (IEEE).
- **Measurable:** Make NFRs quantifiable.
- **Trace:** Maintain the RTM to prevent scope creep.
- **Iterate:** The SRS is a living document.



ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena

Thank You



UNIVERSITY OF
MORATUWA

ITE2953 – Programming
Group Project

Requirement Analysis &
SRS

Savin Gunawardena